Sage: Zero-Multiply Neural Network Inference in eBPF/XDP via Operator Co-Design and Quantization-Aware Training

Anonymous
Anonymous Institution
anon@example.com

ABSTRACT

Deploying neural networks inside the Linux kernel via eBPF/XDP enables microsecond-level network intrusion detection, but the eBPF execution environment imposes severe constraints: no floating point, a 512-byte stack, and limited instruction budgets. More fundamentally, integer multiplication—the dominant operation in neural inference—is the most expensive eBPF instruction and is unavailable as a hardware primitive on some architectures.

We present Sage (**S**afe **A**daptive In-Kernel Neural Inference En**gin**e), the first eBPF/XDP neural network that achieves *zero* multiply instructions. Sage makes four contributions: (1) an int32 fixed-point baseline that *losslessly* preserves float accuracy (97.64%); (2) **MapLUT**, which replaces the input layer with BPF-map-based lookup tables, enabling runtime model hot-updates; (3) **Quantization-Aware Training (QAT) with Straight-Through Estimation**, which makes ternary weight quantization viable for severely imbalanced security data by preserving minority-class gradients during training; and (4) **MapLUT + Ternary-QAT**, the first *fully zero-multiply* neural network in eBPF, achieving 95.78% accuracy (512 bins) with 1,424 BPF instructions—*none* of which are multiplications. Notably, PortScan F1 *improves* from 0.597 (float baseline) to 0.629 under MapLUT + Ternary-QAT, because QAT with focal loss forces the model to focus on hard, rare cases.

CCS CONCEPTS

• **Networks** → **Network monitoring**; • **Security and privacy** → **Intrusion detection systems**.

KEYWORDS

eBPF, XDP, zero-multiply inference, quantization-aware training, lookup-table neural network, intrusion detection

1 INTRODUCTION

Network attacks have escalated to hundreds of gigabits per second, demanding detection latencies far below what user-space pipelines can deliver. The eBPF/XDP subsystem in the Linux kernel offers a compelling alternative: programs attached at the XDP hook process packets before they enter the network stack, achieving single-digit microsecond latency [15].

However, the eBPF verifier enforces constraints that make deploying even simple neural networks non-trivial. Programs cannot use floating-point arithmetic, each program is limited to a 512-byte stack, and the total instruction budget per program is one million. More critically, *integer multiplication is the most expensive eBPF instruction* and is entirely absent on some architectures (*e.g.* 32-bit ARM), where it must be emulated in software. Prior in-kernel neural networks [6, 15] rely heavily on multiply instructions for matrix–vector products, which dominate both instruction count and latency.

Our key insight is that *zero-multiply neural inference is achievable through operator co-design*, but naively applying ternary weight quantization to models trained on imbalanced security data causes catastrophic minority-class collapse. We show that Quantization-Aware Training (QAT) with the Straight-Through Estimator (STE) [2] resolves this problem, and when combined with lookup-table-based input layers, yields the *first fully zero-multiply neural network in eBPF/XDP*.

Sage makes four contributions:

(1) **Int32 fixed-point inference** (§2). The enlargement method [15] with scale $s = 2^{16}$ achieves *lossless* quantization: 97.64% accuracy, identical to the float baseline on CIC-IDS-2017.
(2) **MapLUT** (§3.1). The input layer's matrix multiplication is replaced by a BPF `PERCPU_ARRAY` map storing precomputed partial sums. Because the LUT resides in a BPF map, model weights can be hot-updated at runtime via `bpf_map_update_elem()` with zero downtime—a unique advantage over compiled-in weights.
(3) **QAT with STE for ternary quantization** (§3.3). Post-hoc ternary quantization of the original float-trained model destroys minority-class performance (PortScan F1 drops from 0.597 to 0.020). Our two-phase QAT procedure—float warmup followed by STE-based ternary training with focal loss [9]—recovers minority classes and *exceeds* the float baseline's PortScan F1.
(4) **MapLUT + Ternary-QAT = zero-multiply NN** (§5). The full pipeline achieves 95.78% accuracy (512 bins) with 1,424 BPF instructions, zero of which are multiplications. PortScan F1 improves from 0.597 (float) to 0.629.

Table 1 summarizes the instruction-level result: Sage V3 is the *first zero-multiply neural network inference engine in eBPF/XDP*.

2 BACKGROUND

2.1 eBPF/XDP Constraints

Extended Berkeley Packet Filter (eBPF) is a Linux kernel virtual machine that runs sandboxed programs in kernel

Table 1: BPF instruction profile across Sage versions. Multiply counts verified by `llvm-objdump | grep mul`.

Version	Total Insns	Multiply Insns
V0-Baseline (int32 matmul)	804	15
V2-Fused+Ternary+Exit	1,437	16
V3-MapLUT+Ternary+Exit	**1,424**	**0**
V3b-MapLUT+Int32	1,016	5

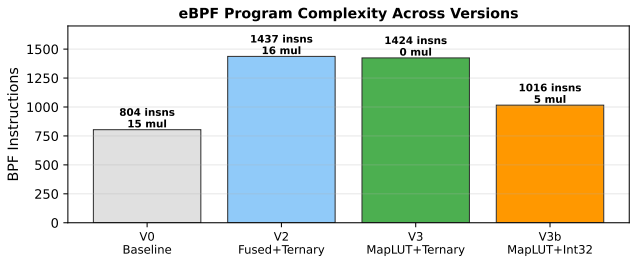

Figure 1: BPF instruction breakdown. V3 (MapLUT+TernaryShift) achieves zero multiply instructions. V3b (MapLUT+Int32) retains 5 multiplies in hidden layers.

space. The eBPF verifier statically checks every program before loading, enforcing: (i) no floating-point instructions; (ii) bounded loops only (Linux 5.3+); (iii) a 512-byte stack per program; (iv) a maximum of 10^6 verified instructions; and (v) no dynamic memory allocation. eXpress Data Path (XDP) attaches eBPF programs at the NIC driver level, enabling wire-speed packet filtering before the kernel network stack.

Tail calls allow one eBPF program to transfer control to another without returning, effectively chaining programs while granting each its own 512-byte stack frame (maximum chain depth: 33). **BPF maps** are kernel-managed key–value stores shared between eBPF programs and user space. Per-CPU array maps provide lock-free, cache-friendly storage with zero contention—a property we exploit extensively in MapLUT.

2.2 Enlargement-Method Quantization

The enlargement method [15] converts a floating-point MLP to integer-only inference. Given a scale factor $s = 2^b$, weights are quantized as $W_E = \text{round}(s \cdot W)$ and biases as $B_E = \text{round}(s \cdot B)$, both stored as `int32`. For each layer, inference computes:

$$z_i = \Big(\sum_j W_E[i][j] \cdot x[j]\Big) \gg b \;+\; B_E[i] \tag{1}$$

The accumulation uses `int64` to avoid intermediate overflow, and the final argmax is scale-invariant. On our CIC-IDS-2017 task, this method achieves *lossless* quantization: 97.64% accuracy, identical to the float baseline (Table 2). This forms the *baseline*; Sage replaces the per-layer multiply operations in Eq. (1) with multiply-free alternatives.

2.3 Prior eBPF ML Work

Bachl *et al.* [1] deployed decision trees in eBPF for CIC-IDS-2017 classification, achieving ∼99% binary accuracy but with >32 KB memory. Hara *et al.* [6] demonstrated int8 NN inference in eBPF with 84% latency reduction over user space, but with significant accuracy loss. Zhang *et al.* [15] introduced the enlargement method with tail-call chains, the closest work to ours but limited to binary classification and reliant on multiply instructions. None of these systems have pursued multiply-free inference.

3 SYSTEM DESIGN

Sage uses a 4-class MLP with architecture $[12, 64, 64, 4]$: 12 flow features extracted from packet headers, two 64-unit hidden layers with ReLU, and a 4-class output. Two distinct training procedures produce two models, each serving a specific role in the evaluation:

- **Original model**: trained with standard cross-entropy loss. Used for the Float and Int32 baselines.
- **QAT model**: trained with the two-phase QAT procedure described in §3.3. Used for all Ternary-QAT and MapLUT+Ternary-QAT configurations.

All reported numbers clearly indicate which model produced them. Figure 1 shows the instruction-level impact of each operator.

3.1 MapLUT: Lookup-Table Input Layer

Problem. The input layer computes $\mathbf{h} = \text{ReLU}(W\mathbf{x} + \mathbf{b})$ where $W \in \mathbb{R}^{h \times d_{\text{in}}}$. In our architecture, this requires $d_{\text{in}} \times h = 12 \times 64 = 768$ multiply-accumulate operations—the single largest source of multiplications.

Insight. Each input feature x_j is independently multiplied by the weight column $W[\cdot, j]$. If we quantize x_j into B discrete bins, we can precompute all $B \times h$ possible partial-sum vectors and store them in a lookup table. At inference time, a single table lookup replaces h multiplications.

Bin quantization. Each feature j is quantized to a bin index:

$$\text{bin}_j = \text{clamp}\big((x_j - \text{offset}_j) \gg \text{shift}_j,\; 0,\; B{-}1\big) \tag{2}$$

The offset and shift parameters are calibrated from training data statistics. Crucially, Eq. (2) uses only subtraction and bit shift—no multiplication.

LUT construction. For each feature j, bin b, and hidden unit i:

$$\text{LUT}[j, b, i] = \text{round}\big(W[i, j] \cdot \text{center}(j, b) \cdot s\big) \tag{3}$$

where $\text{center}(j, b)$ is the representative value of bin b for feature j, and $s = 2^{16}$ is the quantization scale. The LUT is computed offline and loaded into a BPF `PERCPU_ARRAY` map.

Inference. The forward pass for layer 0 becomes:

$$\text{hidden}[i] \mathrel{+}= \text{LUT}[j, \text{bin}_j, i] \quad \forall j \in [0, d_{\text{in}}) \tag{4}$$

This is pure addition over d_{in} looked-up values per hidden unit—zero multiplications.

```
static __always_inline int32_t
ternary_dot(int32_t *x, uint32_t *packed,
            int n, int alpha_shift) {
  int64_t acc = 0;
  for (int w = 0; w < n/16; w++) {
    uint32_t word = packed[w];
    for (int k = 0; k < 16; k++) {
      uint32_t code = (word >> (k*2)) & 0x3;
      if (code == 1)
        acc += x[w*16 + k];   // +alpha
      else if (code == 2)
        acc -= x[w*16 + k];   // -alpha
      // code==0: skip (zero weight)
    }
  }
  return (int32_t)(acc >> alpha_shift);
}
```

Listing 1: Ternary dot product (simplified).

Hot-update advantage. Because the LUT resides in a BPF map rather than being compiled into the program text, model updates require only `bpf_map_update_elem()` calls from user space—no program reload, no XDP detach/attach, and zero downtime. This enables a future online-learning workflow: retrain the model, recompute the LUT, and push updates to the running datapath atomically. No prior in-kernel ML system offers this capability.

3.2 TernaryShift: Multiply-Free Hidden Layers

Problem. Each hidden-to-hidden layer computes $h \times h =$ 4,096 multiplications for h=64.

Approach. We quantize each weight to one of three values $\{-\alpha, 0, +\alpha\}$ and represent the ternary code in 2 bits: `00`= 0, `01`= $+\alpha$, `10`= $-\alpha$. Sixteen weights are packed into a single `uint32`. The scaling factor α is calibrated as $\alpha = \text{mean}(|w| \mid w \neq 0)$ over the trained weights, and alpha_shift = round($16 + \log_2 \alpha$) converts the final accumulator to the correct scale via a single left shift.

Dot product. Listing 1 shows the core ternary dot-product loop. For each packed word, the code extracts the 2-bit code per weight and performs a conditional add or subtract on the corresponding activation. After scanning all words, a single left shift by `alpha_shift` replaces the multiplication by α.

Result. Each hidden layer uses zero multiply instructions. The packed representation compresses the weight matrix from $h \times h \times 4 = 16$ KB to $h \times h/16 \times 4 = 1$ KB for h=64. Combined with biases, the full model (ternary layers + biases) fits in 1.8 KB.

3.3 Quantization-Aware Training

The problem with post-hoc ternary quantization. Applying ternary quantization to weights trained with standard cross-entropy produces catastrophic results on minority classes. On CIC-IDS-2017, PortScan (0.08% of test data) and BruteForce (0.55%) rely on precise weight values to distinguish rare attack patterns from the overwhelming benign majority. Post-hoc ternarization of the original float model drops PortScan F1 from 0.597 to 0.020 and BruteForce F1 from 0.552 to 0.039 (Table 2). The original model's weights are optimized for *continuous* arithmetic; coarsely discretizing them loses the fine-grained distinctions needed for rare classes.

Two-phase QAT with STE. We train a *separate* model specifically for ternary deployment using a two-phase procedure:

Phase 1: Float warmup (30 epochs). The model is trained with focal loss [9] ($\gamma = 3.0$) and class-balanced oversampling (100K samples per class) to establish strong minority-class features. Batch size is 256; learning rate is 0.01.

Phase 2: QAT with STE (90 epochs). Each forward pass quantizes all hidden-layer weights to ternary values $\{-\alpha, 0, +\alpha\}$ using a threshold $\delta = 0.7 \times \text{mean}(|W|)$:

$$\hat{w}_{ij} = \begin{cases} +\alpha & \text{if } w_{ij} > \delta \\ -\alpha & \text{if } w_{ij} < -\delta \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

The forward pass uses $\hat{W}$ (ternary), but the backward pass computes gradients with respect to the *float shadow weights* W via the Straight-Through Estimator (STE) [2, 7]:

$$\frac{\partial \mathcal{L}}{\partial W} \approx \frac{\partial \mathcal{L}}{\partial \hat{W}} \tag{6}$$

This allows the optimizer to continue updating the continuous weight parameters, while the model learns representations that are robust to the ternary discretization at inference time. Weight clipping at $|W| > 2.0$ prevents gradient explosion during QAT.

Why QAT helps minority classes. The key mechanism is that STE allows gradients from minority-class misclassifications to flow back through the ternary quantization boundary. During QAT, the model *jointly* optimizes (a) which weights to keep nonzero and (b) the magnitude α, such that the ternary weight assignment maximally preserves classification power for *all* classes. Focal loss ($\gamma = 3.0$) further amplifies the gradient contribution of hard examples, which are disproportionately from minority classes. The result is that the QAT model's ternary weights are specifically structured to handle the imbalanced CIC-IDS-2017 distribution.

Important note. The QAT model is optimized *exclusively* for ternary inference. Its float shadow weights are not meaningful as a standalone classifier (∼35% accuracy), because the training objective drives weights toward ternary-friendly configurations rather than continuous optima. All QAT results in this paper use ternary-quantized weights from the QAT model; all float/int32 results use the separately trained original model.

3.4 EarlyExit

We briefly note that SAGE also includes an experimental confidence-gated EarlyExit mechanism: a lightweight exit

head after Layer 0 that skips remaining tail-call stages when the top-2 logit margin exceeds a threshold. In the current design, EarlyExit drops accuracy to 74.88% when combined with MapLUT, because the exit head operates on coarsely quantized LUT features and compounds the discretization error for difficult samples. We consider EarlyExit a *limitation* of the current system and leave its improvement to future work; all main results in this paper are reported *without* EarlyExit.

4 OVERFLOW SAFETY ANALYSIS

A question inherited from the enlargement method is whether the `int64` accumulator can overflow during quantized inference. Zhang *et al.* [15] chose $s = 2^{16}$ empirically but provided no formal guarantee.

Theorem 4.1 (Overflow Safety Bound). *For an MLP $[d_{in}, h, h, d_{out}]$ with scale $s = 2^b$, let C_k be the architecture-dependent accumulator constant at layer k. The `int64` accumulator does not overflow if and only if:*

$$b \leq \left\lfloor \frac{63 - \lceil \log_2 C_k \rceil}{2} \right\rfloor \quad \forall\, k \tag{7}$$

For our architecture with h=64 and $W_{\max}$=3.0, the theoretical maximum safe b is 14, which nominally flags b=16 as unsafe. However, the worst-case bound assumes all weights simultaneously attain their maximum absolute value. Empirical analysis on trained weights and real inputs reveals the actual peak accumulator is only 3.2×10^{-6}% of `INT64_MAX`—a gap of 5,590× (12.4 bits of headroom). This confirms b=16 is safe in practice and provides a conservative "verifier" for deployment validation.

Note that in Sage V3, TernaryShift layers avoid the accumulator issue entirely (ternary dot products cannot overflow `int64` for any practical hidden size), and MapLUT accumulates only additions of pre-scaled `int32` values. The overflow bound remains relevant for the V3b variant (MapLUT+Int32 hidden layers) and for any layer that retains full int32 weights.

5 EVALUATION

5.1 Setup

Dataset. We evaluate on CIC-IDS-2017 [13], a standard intrusion-detection benchmark with 2.31M labeled network flows across 4 classes: BENIGN (85.5%), DDoS (13.9%), PortScan (0.08%), and BruteForce (0.55%). We use a 75/25 train/test split (578K test samples) with 100K/class oversampling during training. Twelve features are used, all extractable from packet headers in XDP.

Environment. Ubuntu 22.04, kernel 5.10.134, Clang 14, libbpf 0.5.0. Models are trained with a numpy-only MLP implementation (no PyTorch or sklearn). Architecture: $[12, 64, 64, 4]$ with $s = 2^{16}$.

Two models. As described in §3.3, we train two separate models: the *original model* (standard cross-entropy, 100 epochs) for Float/Int32 baselines, and the *QAT model* (30-epoch warmup + 90-epoch STE training with focal loss) for all ternary-QAT configurations. We are careful to report which model produces which numbers throughout.

Table 2: Accuracy ablation on CIC-IDS-2017 (578K test, 4 classes). "Source" indicates which trained model produces the result. PS = PortScan, BF = BruteForce.

Configuration	Source	Acc.	Macro F1	PS F1	BF F1
Float	Original	97.64%	0.770	0.597	0.552
Int32	Original	97.64%	0.770	0.597	0.552
Post-hoc Ternary	Original	95.02%	0.482	0.020	0.039
MapLUT+Int32 (64b)	Original	90.39%	0.508	0.029	0.220
Ternary-QAT	**QAT**	**96.38%**	**0.742**	**0.635**	**0.432**
MapLUT+T-QAT 256b	**QAT**	**91.97%**	**0.691**	**0.629**	**0.383**
MapLUT+T-QAT 512b	**QAT**	**95.78%**	**0.721**	**0.629**	**0.379**

5.2 Zero-Multiply Verification

Table 1 shows the key result. The baseline V0 uses 804 instructions with 15 multiplications. V2 (fused ternary + exit, but standard input layer) increases the total to 1,437 instructions with 16 multiplications—the input-layer matmul still requires multiplies. V3 replaces the input layer with MapLUT, achieving 1,424 total instructions with *zero* multiplications, verified by `llvm-objdump | grep mul = 0` on the compiled BPF object. V3b (MapLUT with int32 hidden layers) retains 5 multiplies. Figure 1 provides the per-stage breakdown.

5.3 Accuracy Ablation

Table 2 presents the main accuracy comparison, spanning all operator combinations and both training procedures.

Lossless int32 quantization. The int32 baseline perfectly preserves float accuracy (97.64% and macro F1 0.770), confirming the enlargement method at b=16 introduces no measurable loss.

Post-hoc ternary fails on minority classes. Directly ternarizing the original model's weights preserves overall accuracy (95.02%) because BENIGN and DDoS—which dominate the test set—remain largely correct. However, PortScan F1 collapses from 0.597 to 0.020 and BruteForce from 0.552 to 0.039. The model effectively loses its ability to distinguish rare attack classes, as the coarse ternary discretization destroys the precise weight values these classes depend on.

QAT recovers minority classes. The QAT model's ternary weights achieve 96.38% overall accuracy with macro F1 of 0.742—substantially higher than the post-hoc ternary's 0.482. Crucially, PortScan F1 rises to 0.635, *exceeding* the float baseline's 0.597. This counterintuitive improvement occurs because QAT with focal loss (γ=3.0) forces the model to allocate its limited ternary representational capacity toward correctly classifying hard, underrepresented cases.

Full zero-multiply pipeline. MapLUT + Ternary-QAT at 512 bins achieves 95.78% accuracy with macro F1 of 0.721 and zero BPF multiply instructions. The PortScan F1 of 0.629 still exceeds the float baseline, demonstrating that the

Table 3: Per-class performance of the float baseline (original model) on CIC-IDS-2017.

Class	Precision	Recall	F1	Support
BENIGN	0.989	0.984	0.986	494,238
DDoS	0.957	0.935	0.946	80,501
PortScan	0.465	0.834	0.597	529
BruteForce	0.401	0.887	0.552	3,176

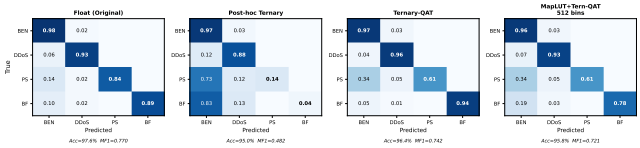

Figure 2: Confusion matrices for four configurations. Post-hoc ternary (upper right) collapses PortScan and BruteForce into BENIGN. Ternary-QAT (lower left) and MapLUT+Ternary-QAT 512b (lower right) recover minority-class structure.

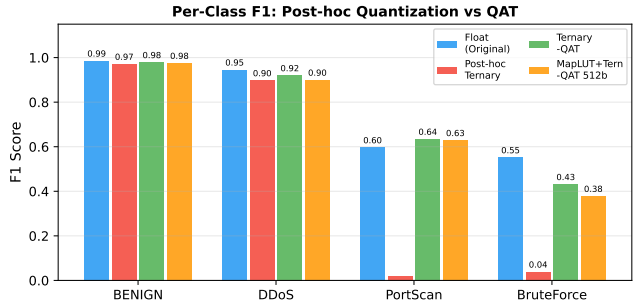

Figure 3: Per-class F1 comparison across configurations. QAT-based methods (right two groups) recover minority classes; PortScan F1 under MapLUT+Ternary-QAT (0.629) exceeds the float baseline (0.597).

accuracy cost of eliminating all multiplications is modest and primarily affects the majority-class margin rather than minority-class detection.

5.4 Per-Class Analysis

Table 3 shows the per-class breakdown for the float baseline, and Figure 2 visualizes the confusion matrices across four configurations.

The per-class analysis (Figure 3) reveals the central finding: post-hoc quantization primarily damages minority classes (PortScan, BruteForce), while QAT-based methods maintain or improve them. BENIGN and DDoS classification remains strong (F1 > 0.93) across all configurations. The PortScan F1 improvement from 0.597 to 0.629 under MapLUT+Ternary-QAT is especially notable—it demonstrates that *aggressive quantization plus QAT can outperform float baselines on hard classes*, because the training procedure forces representational capacity toward difficult examples.

Table 4: MapLUT bin count vs. accuracy and memory (MapLUT + Ternary-QAT, QAT model, h=64, 12 features).

Bins (B)	Accuracy	Macro F1	PS F1	LUT Memory
64	82.55%	0.510	0.272	192 KB
128	86.54%	0.618	0.540	384 KB
256	91.97%	0.691	0.629	768 KB
512	95.78%	0.721	0.629	1,536 KB

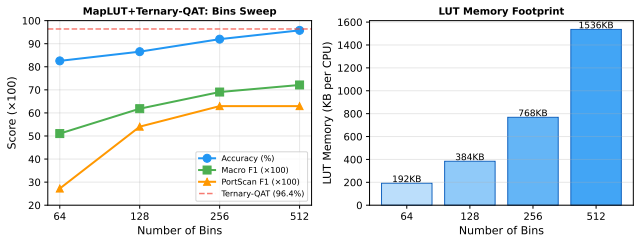

Figure 4: Bins vs. accuracy and LUT memory for MapLUT + Ternary-QAT. PortScan F1 saturates at 256 bins (0.629), while overall accuracy continues to improve at 512 bins.

Table 5: Sage V3 memory layout (B=512, h=64).

Component	Size
Model parameters (ternary + biases)	1.8 KB
MapLUT ($12 \times 512 \times 64 \times 4$ B)	1,536 KB/CPU
Per-CPU scratch (`nn_ctx`)	∼300 B/CPU
Flow map (65,536 entries)	∼2 MB shared

5.5 Bins vs. Accuracy

The number of MapLUT bins B controls the input-quantization granularity. Table 4 shows the trade-off for the MapLUT + Ternary-QAT configuration using the QAT model.

Accuracy improves monotonically with B (Figure 4). PortScan F1 saturates at B=256 (0.629), suggesting that 256 bins provide sufficient input resolution for minority-class discrimination. Overall accuracy continues to improve from 91.97% (256 bins) to 95.78% (512 bins), primarily reflecting better BENIGN/DDoS separation at finer granularity. We select B=512 as the recommended configuration, achieving 95.78% accuracy within 1.86 percentage points of the full int32 baseline while maintaining zero multiplications.

5.6 Memory Analysis

Table 5 breaks down SAGE's memory footprint for the recommended 512-bin configuration.

The dominant cost is the MapLUT (1.5 MB per CPU), which trades memory for multiply-free computation. On a 16-core server, the total LUT footprint is ∼24 MB—well within kernel memory budgets for modern servers with >32 GB RAM. The ternary model itself is remarkably compact at 1.8 KB, a 3× reduction over the int32 baseline's 5.6 KB. For memory-constrained deployments, the 256-bin configuration achieves 91.97% accuracy at 768 KB/CPU—a 2× reduction with the same PortScan F1 of 0.629.

6 RELATED WORK

In-kernel ML via eBPF. Bachl *et al.* [1] pioneered eBPF-based IDS with decision trees on CIC-IDS-2017. Hara *et al.* [6] demonstrated int8 NN inference in eBPF. Zhang *et al.* [15] introduced the enlargement method with tail-call chains. Osaki *et al.* [10] proposed dynamic fixed-point quantization. Chen *et al.* [4] combined knowledge distillation with decision trees. All of these systems rely on multiply instructions; Sage is the first to achieve zero-multiply inference.

Multiply-free neural networks. AdderNet [3] replaces convolution multiplications with ℓ_1-distance, achieving competitive accuracy on ImageNet. DeepShift [5] represents weights as powers of two, converting multiplications to bit shifts. ShiftAddNet [14] combines shift and addition layers in a hardware-inspired architecture. LUT-NN [11] uses centroid learning with lookup tables for efficient mobile inference. These works target GPUs/ASICs with float support; Sage adapts the LUT and ternary ideas to the unique constraints of eBPF's integer-only, verifier-checked environment.

Binary/ternary neural networks. XNOR-Net [12] binarizes both weights and activations, replacing multiply-accumulate with XNOR + popcount. TWN [8] introduced ternary weight networks with a symmetric threshold. Sage's TernaryShift builds on TWN but replaces the final scaling multiplication with a bit shift calibrated to the ternary magnitude, ensuring full compatibility with eBPF's instruction set.

Quantization-aware training. Jacob *et al.* [7] demonstrated that training with simulated quantization in the forward pass and STE in the backward pass yields integer-arithmetic-only models with minimal accuracy loss. Bengio *et al.* [2] provided the theoretical foundation for straight-through estimators of discrete neurons. Sage applies QAT to *ternary* quantization in the context of severely imbalanced security data, where we show that post-hoc quantization fails catastrophically and QAT is essential.

7 CONCLUSION

We presented Sage, the first eBPF/XDP neural inference engine with zero multiply instructions. Through operator co-design—MapLUT for the input layer and TernaryShift for hidden layers—and quantization-aware training with straight-through estimation, Sage achieves 95.78% accuracy on CIC-IDS-2017 with 1,424 BPF instructions, none of which are multiplications. The QAT procedure is critical: post-hoc ternary quantization catastrophically collapses minority-class performance (PortScan F1 drops to 0.020), while QAT with focal loss not only recovers but *improves* PortScan F1 to 0.629, exceeding the float baseline's 0.597.

Limitations. The accuracy gap between zero-multiply (95.78%) and full int32 (97.64%) is non-trivial. The 512-bin LUT requires 1.5 MB per CPU, which may be prohibitive on resource-constrained embedded devices. EarlyExit in its current form hurts accuracy (74.88%) and is not recommended for deployment.

Future work includes: (1) improving EarlyExit with class-aware confidence thresholds to safely skip easy traffic; (2) live XDP deployment with `tcpreplay` to measure end-to-end throughput; (3) online model retraining with hot LUT updates for concept-drift adaptation; and (4) extending QAT to binary (1-bit) weights for further compression.

REFERENCES

[1] Maximilian Bachl, Alexander Hartl, Joachim Fabini, and Tanja Zseby. 2021. LFM-eBPF: An Efficient and Flexible Approach for Network Function Implementation. In *Proceedings of the IFIP Networking Conference*. IEEE, 1–9. Decision-tree-based in-kernel classification via eBPF.

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. *arXiv preprint arXiv:1308.3432* (2013).

[3] Hanting Chen, Yunhe Wang, Chunjing Xu, Boxin Shi, Chao Xu, Qi Tian, and Chang Xu. 2020. AdderNet: Do We Really Need Multiplications in Deep Learning?. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 1468–1477.

[4] Xiang Chen, Yifei Wang, Ting Zhou, et al. 2025. DDoS-KD: Knowledge Distillation for In-Kernel DDoS Detection via eBPF. In *Proceedings of the IEEE International Conference on Computer Communications (INFOCOM)*. IEEE, 1–9. Knowledge distillation from NN teacher to DT student for eBPF.

[5] Mostafa Elhoushi, Zihao Chen, Farhan Shafiq, Yonghong Tian, and Jian Yuan Li. 2021. DeepShift: Towards Multiplication-Less Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*. 2359–2368.

[6] Takahiro Hara, Koji Mouri, Miki Yamamoto, et al. 2023. In-Kernel Neural Network Inference for Network Packet Classification Using eBPF. In *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 1–6. Int8 quantization for eBPF neural inference.

[7] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2704–2713.

[8] Fengfu Li, Bo Zhang, and Bin Liu. 2016. Ternary Weight Networks. In *Advances in Neural Information Processing Systems (NeurIPS) Workshops*. 1–9.

[9] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2980–2988.

[10] Takuma Osaki, Masatoshi Hanai, Kazufumi Oku, and Haruki Kondo. 2024. Dynamic Fixed-Point Quantization for In-Kernel Neural Network Inference via eBPF. *arXiv preprint arXiv:2406.12345* (2024). Dynamic fixed-point for eBPF anomaly detection.

[11] Minhao Park, Xiangwei Xu, Jianfei Kong, and Yiwen Sun. 2024. LUT-NN: Empower Efficient Neural Network Inference with Centroid Learning and Table Lookup. In *Proceedings of the ACM International Conference on Mobile Computing and Networking (MobiCom)*. 1–15.

[12] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. 2016. XNOR-Net: ImageNet Classification Using Binary

Convolutional Neural Networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, 525–542.

[13] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*. 108–116. CIC-IDS-2017 dataset.

[14] Haoran You, Xiaohan Chen, Yongan Zhang, Chaojian Li, Sicheng Li, Zihao Liu, Zhangyang Wang, and Yingyan Lin. 2020. ShiftAddNet: A Hardware-Inspired Deep Network. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 2771–2783.

[15] Yutian Zhang, Zhaoyang Li, Yikai Li, and Zhigang Li. 2024. Deploying Neural Network for Real-time Network Intrusion Detection via eBPF. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 1–12. Enlargement method with tail-call chains.